

ADAPTIVE DEEP NEURAL NETWORK INFERENCE USING CONFIDENCE-BASED EARLY EXITS

**Final Project Report on
Deep Learning Project
[ICT-4442]**

Submitted By

**Aditya Goyal | 230953554
Raunak Panshikar | 230953326
Arth Mishra | 230911568
Kartikey Goyal | 230911536**



MANIPAL
ACADEMY *of* HIGHER EDUCATION
(Institution of Eminence Deemed to be University)

**SCHOOL OF COMPUTER ENGINEERING
MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL ACADEMY OF HIGHER EDUCATION**

OCTOBER 2026

Team No.: 15

Aditya Goyal (230953554)

Raunak Panshikar (230953326)

Arth Mishra (230911568)

Kartikey Goyal (230911536)

Code repository: <https://github.com/goyaladitya05/adaptive-dnn-inference>**Abstract**

Early-exit networks attach classifiers at several depths of a backbone so that easy inputs can stop early. We study confidence-based early exiting on CIFAR-100 with a ResNet-18 baseline, a confidence-based early-exit ResNet-18, a calibrated and entropy-based variant of the same network, and an early-exit ViT-Tiny, trained with self-distillation and averaged over three seeds. With thresholds chosen on validation so that no accuracy is lost relative to its own final exit, the early-exit ResNet-18 reaches $75.32 \pm 0.12\%$ (ResNet-18: $76.07 \pm 0.31\%$) with 51% fewer FLOPs, and per-exit thresholds raise the saving to 55%. The savings become measured speed-ups on a CPU (1.90x with one thread) but barely for single images on a T4 GPU (1.06x), where per-exit synchronisation dominates. Temperature scaling lowers the calibration error of every exit from 12-16% to at most 4.1% but makes routing on clean data more expensive. Under corruption the self-distilled CNN exits stay overconfident: at severity 5, 74% of images still exit early and only 17% of those decisions are correct. Calibrated entropy halves the early exits and makes them more accurate, and the underconfident ViT exits become cautious on their own. Early exits thus save substantial computation, but reliable routing under distribution shift needs calibration that holds beyond the clean data.

1. Introduction

Deep neural networks normally execute every layer for every input, although many inputs can be classified correctly by a fraction of the network. Early-exit networks attach intermediate classifiers to a backbone and let an input stop at the first classifier whose prediction is confident enough, so that easy inputs use less computation and difficult inputs receive the full network. This is attractive for latency-sensitive and resource-constrained deployment, but it raises three practical questions that this project studies on CIFAR-100: how much computation can be saved for a given loss in accuracy, whether the confidence used for routing can be trusted (in particular after calibration and under corrupted inputs), and whether savings in FLOPs translate into measured latency.

1.1 Objectives

1. Develop an adaptive image-classification network that dynamically selects inference depth for individual inputs.
2. Quantify the trade-off between classification accuracy, average computation and inference latency for different exit strategies.
3. Evaluate the reliability of confidence-based early exits under clean and corrupted input conditions.

1.2 Contributions

- A common implementation of four models (ResNet-18, confidence-based and calibrated/entropy-based early-exit ResNet-18, early-exit ViT-Tiny) with real early-exit inference that removes exited images from the batch.
- A GPU data pipeline for CIFAR-100 with a re-implementation of six CIFAR-100-C corruptions at five severities.
- A comparison of four exit rules (max-prob and entropy, with and without temperature scaling), with global and per-exit thresholds chosen on validation data, averaged over three training seeds.
- Ablations of self-distillation, exit-head design and exit placement.
- Measured latency on a GPU and a CPU next to FLOP counts, and an analysis of how exit decisions behave under distribution shift.

1.3 Team and Model Ownership

Table 1: Model-to-member assignment.

Member	Model / task owned
Aditya Goyal	Model 2: Confidence-based early-exit ResNet-18
Raunak Panshikar	Model 1: Standard ResNet-18 baseline
Arth Mishra	Model 3: Calibrated / entropy-based early-exit ResNet-18
Kartikey Goyal	Model 4: Early-exit ViT-Tiny

2. Literature Review

We reviewed ten papers covering early-exit CNNs [1], [2], [4], [8], input-dependent routing [3], [6], calibration [5], [10], corruption robustness [7] and early exiting in vision transformers [9]. Table 2 summarises each paper; all quantitative results were checked against the original papers.

Table 2: Summary of reviewed literature.

#	Paper	Method	Dataset(s)	Key result	Relevance to this project
[1]	Teerapittayanon et al., ICPR 2016 (BranchyNet)	Side-branch classifiers on LeNet, AlexNet and ResNet-110, trained jointly; a sample exits at the first branch whose softmax entropy is below a tuned threshold.	MNIST, CIFAR-10	B-ResNet-110 (CIFAR-10): 1.9x measured speed-up on CPU and GPU, accuracy 79.2% vs 80.7%; B-LeNet: 5.4x CPU speed-up, 94% of samples exit early.	Origin of the entropy exit criterion used in Model 3; one of few works reporting measured latency rather than FLOPs.
[2]	Huang et al., ICLR 2018 (MSDNet)	Multi-scale dense network keeping coarse and fine features throughout, with intermediate classifiers; exits when the maximum softmax probability exceeds a threshold.	CIFAR-10, CIFAR-100, ImageNet	CIFAR-100 budgeted batch: matches ResNet-110 accuracy with about 1/10 of its FLOPs; ImageNet anytime: about 4-8% higher accuracy than ResNet/DenseNet ensembles at low budgets.	Standard CIFAR-100 multi-exit baseline; shows intermediate classifiers can reduce a ResNet's final accuracy (up to 7%), a risk for our ResNet-18.
[3]	Wang et al., ECCV 2018 (SkipNet)	Lightweight gates skip residual blocks per input; trained with supervised pre-training followed by hybrid reinforcement learning.	CIFAR-10, CIFAR-100, SVHN, ImageNet	Computation reduced by 30-90% at preserved accuracy; for the deepest ResNets, savings of 37% (CIFAR-100), 50% (CIFAR-10), 86% (SVHN), 30% (ImageNet).	Layer-skipping alternative to early exiting; positions our exit-based design among dynamic-depth methods for ResNets.
[4]	Kaya et al., ICML 2019 (SDN)	Internal classifiers added to VGG-16, ResNet-56, WRN-32-4 and MobileNet; exits on maximum softmax confidence; characterises network "overthinking".	CIFAR-10, CIFAR-100, Tiny ImageNet	Confidence exits cut average inference cost (FLOPs) by over 50% on CIFAR-10/100 while preserving accuracy; VGG-16-SDN on CIFAR-100: 72.5% at $\leq 50\%$ cost vs 70.9%.	Closest precedent for Model 2: the same max-softmax exit rule on CIFAR-100 ResNets.
[5]	Guo et al., ICML 2017	Study of calibration in modern networks; compares histogram binning, isotonic regression, BBQ, and matrix, vector and temperature scaling as post-hoc fixes.	CIFAR-10/100, ImageNet, SVHN, CUB, Cars, 20 News, Reuters, SST	Modern networks are poorly calibrated; temperature scaling reduces ResNet-110 CIFAR-100 ECE from 16.53% to 1.26% without changing accuracy.	Provides ECE and temperature scaling, applied per exit in Model 3 so that one threshold means the same at every exit.
[6]	Wu et al., CVPR 2018 (BlockDrop)	A policy network trained with reinforcement learning decides in one shot which residual blocks of a pretrained ResNet to execute per image.	CIFAR-10, CIFAR-100, ImageNet	ResNet-101 on ImageNet: 20% average speed-up (in FLOPs) at unchanged 76.4% top-1; ResNet-110 on CIFAR-100: 73.7% using 55% of blocks.	Its sequential-policy variant ran slower than the full ResNet despite fewer blocks: FLOP savings need not imply wall-clock savings.
[7]	Hendrycks and Dietterich, ICLR 2019	Robustness benchmark of 15 algorithmic corruptions (noise, blur, weather, digital) at five severities, summarised by mean Corruption Error (mCE).	ImageNet-C/P, CIFAR-10-C, Tiny ImageNet-C (CIFAR-100-C in the same repository)	Relative corruption robustness barely changed from AlexNet to ResNet; ResNet-50 mCE 76.7%, MSDNet 73.6%.	Defines the corruption types and severity constants we re-implement to stress-test exit policies.
[8]	Wolczyk et al., NeurIPS 2021 (ZTW)	Cascade connections pass each internal classifier's output to the next; a weighted geometric ensemble of past predictions feeds the confidence-based exit decision.	CIFAR-10, CIFAR-100, Tiny ImageNet, ImageNet	ResNet-56 on CIFAR-100 at 50% of base FLOPs: 62.1% accuracy vs 57.2% (SDN) and 53.5% (PABEE).	Strong confidence-exit baseline on CIFAR-100; shows discarded early predictions carry information an exit policy can reuse.
[9]	Xu et al., ACM MM 2023 (LGViT)	Early-exit ViTs with heterogeneous heads (local perception, global aggregation), two-stage training with self-distillation, max-softmax exits.	CIFAR-100, Food-101, ImageNet-1K	About 1.8x average speed-up for about 2% accuracy loss; ViT-B/16 on CIFAR-100: 88.5% vs 90.8% at 1.87x (computed from exit layers).	Closest vision-transformer precedent for Model 4; its speed-up is derived from exit depth, not measured latency.
[10]	Meronen et al.,	Post-hoc last-layer Laplace	CIFAR-100,	CIFAR-100 (small MSDNet,	Direct evidence that early-exit

#	Paper	Method	Dataset(s)	Key result	Relevance to this project
	WACV 2024	approximation at each exit plus model-internal ensembling and tuned temperature, for uncertainty-aware exits in MSDNet.	ImageNet, Caltech-256	averaged over budgets): ECE falls from 0.182 to 0.017 and top-1 rises from 69.25% to 69.84%.	confidences are overconfident and that calibrating them helps exit decisions; motivates Model 3.

2.1 Discussion

Early-exit networks. BranchyNet [1] introduced jointly trained side branches and an entropy threshold, reporting a measured 1.9x speed-up for a branchy ResNet-110 on CIFAR-10 at a cost of 1.5 points of accuracy. MSDNet [2] showed that naive intermediate classifiers can lower a ResNet’s final accuracy by up to 7% on CIFAR-100, and addressed this with a multi-scale dense architecture that matches ResNet-110 with about one tenth of the computation. Shallow-Deep Networks [4] attached internal classifiers to off-the-shelf networks, identified overthinking (correct intermediate predictions that later become wrong), and cut average FLOPs by more than half with a max-softmax threshold. Zero Time Waste [8] reuses the predictions of exits that did not fire through cascade connections and ensembling, improving the accuracy of confidence-based exits on CIFAR-100 at a fixed budget.

Dynamic routing. SkipNet [3] and BlockDrop [6] make depth input-dependent by skipping residual blocks using learned gates or a policy network. They report 30-90% and 20% FLOP reductions respectively, but BlockDrop’s own timing shows that a sequential decision variant can run slower than the unmodified network, an early warning that FLOP counts and wall-clock time can disagree.

Calibration and robustness. Guo et al. [5] showed that modern networks are overconfident and that a single temperature fitted on validation data almost removes the calibration error without changing predictions. Meronen et al. [10] found the same overconfidence in early-exit networks and showed that calibrating each exit improves both ECE and accuracy at a given budget. Hendrycks and Dietterich [7] defined the corruption benchmark (15 corruption types, 5 severities) that we follow. LGViT [9] brought early exits to vision transformers, showing that simple heads on shallow transformer blocks are weak and require specialised heads and self-distillation.

2.2 Research Gap

Most early-exit work decides with raw softmax confidence or entropy [1], [2], [4], [8], [9] without an explicit calibration step, although networks, and early exits in particular, are overconfident [5], [10]. Efficiency is mostly reported as FLOPs or exit depth [2]-[4], [8]-[10], although decision overhead can erase theoretical savings [6]. Finally, none of the reviewed early-exit methods studies how exit thresholds, exit distributions and calibration behave when inputs are corrupted [7]. This project addresses all three points on a common CIFAR-100 setup: it compares raw and calibrated exit rules, reports measured latency next to FLOPs, and evaluates every exit policy under noise, blur and contrast corruptions.

3. Dataset Acquisition and Preprocessing Pipeline

3.1 Dataset

CIFAR-100 [11] contains 60,000 RGB images of 32x32 pixels in 100 classes (600 per class), split into 50,000 training and 10,000 test images. It is downloaded programmatically through torchvision, so every run (local or on Kaggle) obtains identical data. We hold out a stratified validation set of 5,000 training images (50 per class, fixed seed 0) that is used for model selection, fitting temperatures and choosing exit thresholds. The official test set is used only for reporting. Figure 1 shows random test samples.

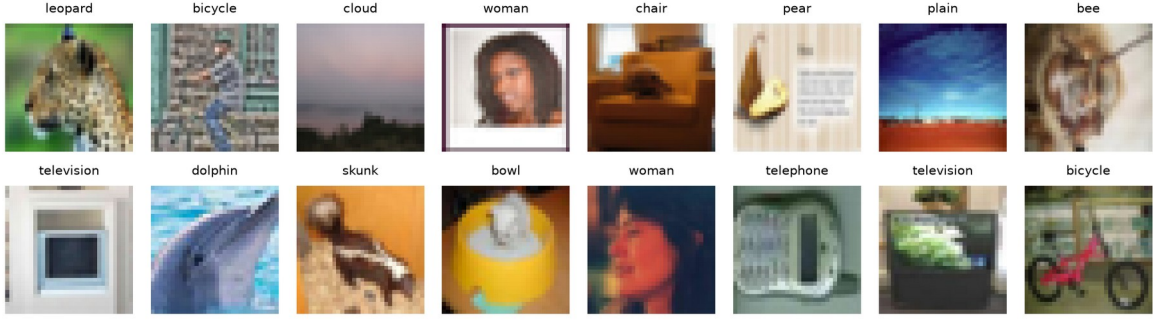


Figure 1: Random CIFAR-100 test images with their class labels.

Table 3: Data splits.

Split	Images	Per class	Used for
Train	45,000	450	Gradient updates
Validation	5,000	50	Monitoring, temperature fitting, threshold selection
Test (clean)	10,000	100	Final accuracy, compute, calibration, latency
Test (corrupted)	6 x 5 x 10,000	100	Robustness of exit policies (Section 3.3)

3.2 Preprocessing and Augmentation

Kaggle machines provide two T4 GPUs but only four CPU cores, so a standard CPU data loader with per-image PIL transforms becomes the bottleneck for small 32x32 images. Instead, the full uint8 dataset (about 150 MB) is copied to GPU memory once, and batches are sampled, augmented and normalised on the GPU. Training batches use the standard light CIFAR augmentation: random 32x32 crops from a 4-pixel reflect-padded image and random horizontal flips. Images are scaled to $[0, 1]$ and normalised with the per-channel CIFAR-100 training mean (0.507, 0.487, 0.441) and standard deviation (0.267, 0.256, 0.276), which we verified on our training split. The ViT-Tiny additionally uses mixup [15] or CutMix [16] on every batch and label smoothing of 0.1, since transformers trained from scratch on small datasets need stronger regularisation. For the final runs the ViT also uses RandAugment (two operations, magnitude 9), applied on the GPU to chunks of 32 images that each draw their own operations. Evaluation uses no augmentation.

```
def load_cifar100(root="data", val_size=5000, seed=0):
    tr = CIFAR100(root, train=True, download=True)
    te = CIFAR100(root, train=False, download=True)
    x, y = tr.data, np.asarray(tr.targets)
    rng = np.random.default_rng(seed)
    per_class = val_size // NUM_CLASSES # 50 images per class
    val_idx = np.concatenate([rng.permutation(np.flatnonzero(y == c))[:per_class]
                              for c in range(NUM_CLASSES)])
    mask = np.ones(len(y), dtype=bool)
    mask[val_idx] = False
    return {"train": (x[mask], y[mask]),
            "val": (x[val_idx], y[val_idx]),
            "test": (te.data, np.asarray(te.targets))}
```

```
def augment(x, pad=4):
    """Random crop with reflect padding and horizontal flip, on the GPU."""
    n, _, h, w = x.shape
    x = F.pad(x, (pad, pad, pad, pad), mode="reflect")
    i = torch.randint(0, 2 * pad + 1, (n, 1), device=x.device) + arange(h)
    j = torch.randint(0, 2 * pad + 1, (n, 1), device=x.device) + arange(w)
    x = x[arange(n)[:, None, None, None], arange(3)[None, :, None, None],
          i[:, None, :, None], j[:, None, None, :]]
    flip = torch.rand(n, device=x.device) < 0.5
    return torch.where(flip[:, None, None, None], x.flip(3), x)
```

```
class GPULoader:
    # the whole uint8 split lives on the GPU
    def __iter__(self):
        idx = torch.randperm(n) if self.train else torch.arange(n)
        for k in range(0, n, self.bs):
            b = idx[k:k + self.bs]
            xb = self.x[b].float().div_(255)
            if self.train:
                xb = augment(xb)
```

```

if self.transform is not None: # corruptions, Section 3.3
    xb = self.transform(xb)
    yield normalize(xb), self.y[b]

```

3.3 Corrupted Test Sets

To study whether exit decisions stay reliable under distribution shift, we re-implemented six corruptions of the CIFAR-100-C benchmark [7] in PyTorch so they run on the GPU: Gaussian, shot (Poisson) and impulse (salt-and-pepper) noise, Gaussian and defocus blur, and contrast reduction, each at five severities with the constants of the original CIFAR-C generation code. Corrupted images are re-quantised to 8 bits, as in the stored benchmark, and every corruption and severity uses a fixed random seed, so all models see identical corrupted images. This gives 30 corrupted copies of the test set (300,000 images) without storing any extra data. Figure 2 shows one test image under each corruption.

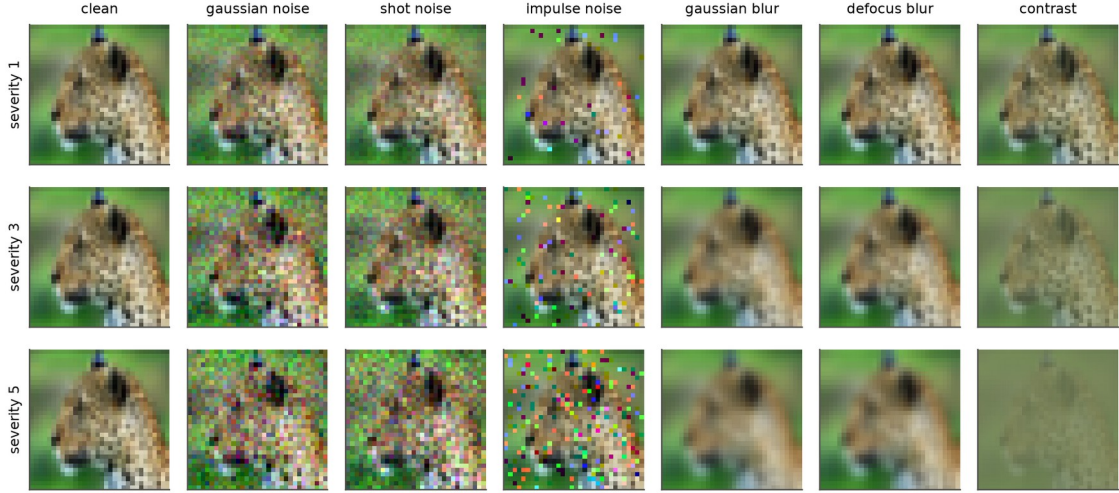


Figure 2: One test image under the six corruptions at severities 1, 3 and 5.

```

def gaussian_noise(x, s, g):
    c = [0.04, 0.06, 0.08, 0.09, 0.10][s - 1]
    return (x + torch.randn(x.shape, generator=g, device=x.device) * c).clamp(0, 1)

def gaussian_blur(x, s, g):
    c = [0.4, 0.6, 0.7, 0.8, 1.0][s - 1]
    k = _gauss1d(c, int(4 * c + 0.5), x.device)
    return _filter(x, torch.outer(k, k), "replicate").clamp(0, 1)

def corrupt(name, severity, generator): # re-quantize to 8 bits like stored CIFAR-C
    fn = CORRUPTIONS[name]
    return lambda x: torch.round(fn(x, severity, generator) * 255) / 255

```

4. Methodology

4.1 Multi-Exit Networks and Early-Exit Inference

Every model is a backbone split into K segments with a classifier head after each segment; a plain network is the case $K = 1$. At inference, an image passes through segment k , head k produces logits and a score, and the image stops if the score clears a threshold. Our implementation removes exited images from the batch before the next segment, so savings are real and not only simulated. Stopping at exit k costs the backbone up to segment k plus every head evaluated so far.

```

@torch.no_grad()
def adaptive_forward(model, x, score, thresh, temps=None):
    """Real early-exit inference: exited samples are dropped before the next segment."""
    alive, h = torch.arange(len(x)), x
    for k in range(model.num_exits):
        h = model.segments[k](h)
        logits = model.heads[k](h)
        if k == model.num_exits - 1:
            preds[alive] = logits.argmax(1); break
        done = score(logits, temps[k]) >= thresh # max_prob or 1 - H(p)/log C

```

```

preds[alive[done]] = logits[done].argmax(1); exit_at[alive[done]] = k
alive, h = alive[~done], h[~done]

```

4.2 Models

Table 4: Architectures used in the final experiments.

Model	Params (M)	Exits	Cumulative GFLOPs at each exit
ResNet-18 (Model 1)	11.22	1	1.111
EE-ResNet-18 (Models 2, 3)	12.22	4	0.367 / 0.659 / 0.937 / 1.205
EE-ResNet-18, pooling heads	11.27	4	0.306 / 0.574 / 0.842 / 1.111
EE-ViT-Tiny (Model 4)	5.44	4	0.181 / 0.360 / 0.539 / 0.719

- **Model 1, ResNet-18 [12].** CIFAR variant: 3x3 stem without max-pooling, four stages of two basic blocks (64 to 512 channels), global pooling and a linear classifier.
- **Models 2 and 3, EE-ResNet-18.** The same backbone with an exit after each stage. Heads for stages 1 to 3 use strided 3x3 convolutions (128 channels) down to 4x4, then pooling and a linear layer; the final exit is the standard classifier. Models 2 and 3 share one trained network and differ only in the exit rule.
- **Ablation, pooling heads.** Every exit is only global pooling and a linear layer, as in SDN [4]. Heads are almost free but see less processed features.
- **Model 4, EE-ViT-Tiny [13], [14].** Width 192, 12 blocks, 3 heads, 4x4 patches (64 tokens), exits after blocks 3, 6, 9 and 12, each using LayerNorm, token averaging and a linear layer.

4.3 Training Objective with Self-Distillation

All exits are trained jointly. The final exit minimises the cross-entropy with the labels. Each earlier exit minimises $(1 - \alpha)$ times its cross-entropy plus α times a distillation term, the KL divergence between the softened prediction of the final exit (temperature τ , gradient stopped) and its own softened prediction, scaled by τ squared. We use $\alpha = 0.5$ and $\tau = 3$, and average the losses over exits. Self-distillation lets shallow exits learn from the richer predictions of the deepest exit, as in LGViT [9]; an ablation trains the same network without it.

```

def multi_exit_loss(outs, ya, yb, lam, ls, alpha=0.0, tau=3.0):
    """Mean over exits; early exits also match the final exit (self-distillation)."""
    teacher = F.softmax(outs[-1].detach().float() / tau, -1)
    total = ce(outs[-1])
    for o in outs[:-1]:
        loss = ce(o)
        if alpha:
            kd = F.kl_div(F.log_softmax(o.float() / tau, -1), teacher,
                          reduction="batchmean") * tau ** 2
            loss = (1 - alpha) * loss + alpha * kd
        total = total + loss
    return total / len(outs)

```

4.4 Exit Rules and Calibration

Max-prob (Model 2) exits when the largest softmax probability is at least a threshold τ . **Entropy** exits when $1 - H(p)/\log(100)$ is at least τ , so it uses the whole predictive distribution, as in BranchyNet [1]. For Model 3, each exit is first calibrated with **temperature scaling** [5]: a scalar T_k is fitted on the validation set by minimising negative log-likelihood and logits are divided by T_k . This keeps every prediction unchanged but places the confidences of all exits on a common, calibrated scale. Calibration is measured with the expected calibration error (ECE, 15 bins).

4.5 Threshold Selection

All thresholds are chosen on the validation set and then frozen. The main operating point uses a **global threshold**, the cheapest single value whose validation accuracy is at least that of the same model's final exit (no accuracy loss on validation). **Per-exit thresholds** start from this value and are refined by coordinate descent, minimising validation cost under the same constraint. A **relaxed** point allows one point of validation accuracy loss. As an optimistic reference we also report the smallest cost at which the

test accuracy reaches that of ResNet-18 (a threshold chosen on test, so an upper bound on the saving). Under corruption, the thresholds chosen on clean validation data are kept, as they would be in deployment.

```
def tune_per_exit(scores, correct, costs, target, init, grid=THRESH, passes=3):
    """Coordinate descent over one threshold per exit: min cost s.t. accuracy >= target."""
    th = np.full(scores.shape[0], float(init))
    best = routing_stats(route(scores, th), correct, costs)
    for _ in range(passes):
        for k in range(len(th)):
            for t in grid:
                cand = th.copy(); cand[k] = t
                r = routing_stats(route(scores, cand), correct, costs)
                if r["acc"] >= target and (r["cost"], -r["acc"]) < (best["cost"], -best["acc"]):
                    th, best = cand, r
    return th.tolist(), best
```

4.6 Metrics

- **Accuracy and compute:** top-1 test accuracy and mean FLOPs per image, counted with PyTorch's FlopCounterMode including every head an image passes.
- **Calibration:** ECE per exit, before and after temperature scaling.
- **Overthinking:** share of test images that some early exit classifies correctly but the final exit does not, and the oracle that stops each image at its earliest correct exit.
- **Latency:** mean time per image at batch 1 on a T4 GPU and on a CPU with 1 and 4 threads, and throughput with batches of 256, measured for all seed-0 models in one Kaggle session.
- **Robustness:** accuracy, mean compute, exit distribution and accuracy of early-exited images on 30 corrupted test sets, and ECE under corruption.

5. Experimental Setup

Table 5: Final training configuration.

Setting	ResNet-18 family	EE-ViT-Tiny
Optimiser	SGD, momentum 0.9, Nesterov	AdamW (0.9, 0.999)
Learning rate	0.1, 2 warmup epochs, cosine	1e-3, 5 warmup epochs, cosine
Weight decay	5e-4 (not on BN / bias)	0.05 (not on norm / bias / position)
Batch size, epochs	128, 200	256, 200
Augmentation	random crop, flip	crop, flip, RandAugment, mixup / CutMix
Regularisation	none	label smoothing 0.1, drop path 0.1, gradient clip 1.0
Self-distillation	alpha 0.5, tau 3 (early-exit models)	alpha 0.5, tau 3
Seeds	0, 1, 2 (ablations: seed 0)	0, 1, 2
Hardware	Kaggle, 2 x Tesla T4, fp16 mixed precision	Kaggle, 2 x Tesla T4, fp16 mixed precision

Eleven training runs were executed in two Kaggle sessions, each running one queue of runs per T4 GPU: three seeds each of ResNet-18, EE-ResNet-18 with self-distillation and EE-ViT-Tiny, plus EE-ResNet-18 without distillation and with pooling heads (seed 0). After training, each run stores the logits of every exit for the validation set, the clean test set and the 30 corrupted test sets, so that all exit rules, thresholds and ablations are evaluated offline from the same predictions. Latency was measured afterwards in a separate session that loads the seed-0 checkpoints, so all models were timed on the same idle machine. Results are reported as mean $\pm$ standard deviation over seeds where three seeds are available.

6. Results

6.1 Training

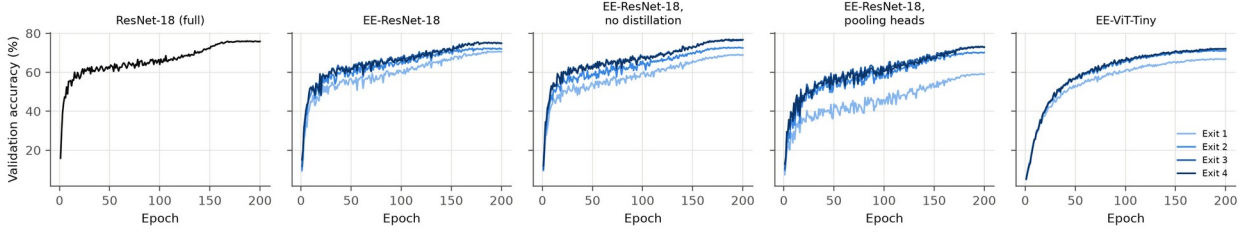


Figure 3: Validation accuracy of every exit during training (seed 0).

All runs converged within 200 epochs. In EE-ResNet-18 the deep exits track the final exit closely during training, and in the ViT the exits after six or more blocks are almost indistinguishable, a first sign that much of each network is redundant for many images. The pooling-head variant learns visibly weaker early exits.

6.2 Accuracy of Each Exit

Table 6: Top-1 test accuracy (%) when every image is forced through a given exit.

Model	Seeds	Exit 1	Exit 2	Exit 3	Final exit
ResNet-18 (Model 1)	3	-	-	-	76.07 ± 0.31
EE-ResNet-18 (Models 2, 3)	3	70.52 ± 0.32	73.06 ± 0.62	76.22 ± 0.27	76.08 ± 0.25
EE-ResNet-18, no distillation	1	67.92	72.21	76.20	76.41
EE-ResNet-18, pooling heads	1	59.13	69.78	72.85	72.77
EE-ViT-Tiny (Model 4)	3	66.41 ± 0.10	70.81 ± 0.21	71.30 ± 0.02	71.32 ± 0.05

ResNet-18 reaches $76.07 \pm 0.31\%$. The final exit of the self-distilled EE-ResNet-18 reaches $76.08 \pm 0.25\%$, the same within one standard deviation, so the early exits do not harm the full network. Exit 1, after the first residual stage at 33% of the baseline cost, reaches $70.52 \pm 0.32\%$, and exit 3 (84% of the cost) already matches the baseline ($76.22 \pm 0.27\%$). EE-ViT-Tiny reaches $71.32 \pm 0.05\%$ at its final exit, about five points more than in the interim study thanks to the longer schedule, RandAugment and distillation, but still below the CNN, as is typical for a transformer trained from scratch on 32x32 images [13], [14]. Its exit 2, after six of twelve blocks, is within 0.5 points of the final exit.

6.3 Accuracy versus Computation

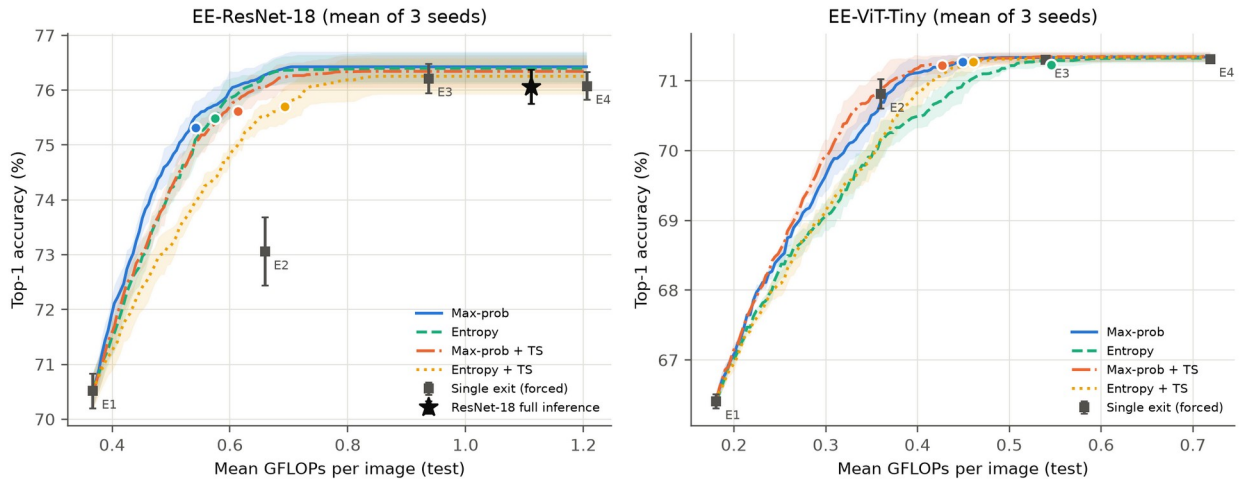


Figure 4: Test accuracy against mean GFLOPs per image while sweeping a global threshold (mean and standard deviation over seeds). Dots are the main operating points selected on validation.

Table 7: EE-ResNet-18 operating points chosen on validation: global and per-exit thresholds with no validation loss, and a relaxed global threshold allowing one point. Savings are relative to ResNet-18 ($76.07 \pm 0.31\%$ at 1.111 GFLOPs).

Exit rule	Thresholds	Test acc. (%)	Mean GFLOPs	Saved (%)	Exits 1/2/3/4 (%)
Max-prob	global	75.32 ± 0.12	0.542 ± 0.037	51.2	65 / 18 / 8 / 10

Exit rule	Thresholds	Test acc. (%)	Mean GFLOPs	Saved (%)	Exits 1/2/3/4 (%)
Max-prob	per-exit	75.08 $\pm$ 0.21	0.504 $\pm$ 0.010	54.7	67 / 21 / 11 / 2
Max-prob	relaxed	74.21 $\pm$ 0.16	0.473 $\pm$ 0.015	57.4	76 / 15 / 5 / 4
Entropy	global	75.48 $\pm$ 0.26	0.574 $\pm$ 0.031	48.3	61 / 18 / 8 / 13
Entropy	per-exit	75.36 $\pm$ 0.21	0.541 $\pm$ 0.016	51.3	64 / 17 / 14 / 5
Entropy	relaxed	74.40 $\pm$ 0.43	0.509 $\pm$ 0.027	54.2	71 / 15 / 6 / 8
Max-prob + TS	global	75.62 $\pm$ 0.17	0.613 $\pm$ 0.066	44.8	58 / 15 / 8 / 19
Max-prob + TS	per-exit	75.47 $\pm$ 0.09	0.552 $\pm$ 0.024	50.3	62 / 17 / 16 / 5
Max-prob + TS	relaxed	74.45 $\pm$ 0.15	0.507 $\pm$ 0.020	54.3	73 / 12 / 6 / 9
Entropy + TS	global	75.70 $\pm$ 0.29	0.693 $\pm$ 0.056	37.7	50 / 14 / 8 / 29
Entropy + TS	per-exit	75.74 $\pm$ 0.24	0.616 $\pm$ 0.029	44.5	50 / 16 / 32 / 3
Entropy + TS	relaxed	74.57 $\pm$ 0.72	0.582 $\pm$ 0.063	47.6	65 / 11 / 6 / 18

Table 8: EE-ViT-Tiny operating points, savings relative to its own full network (71.32 $\pm$ 0.05% at 0.719 GFLOPs).

Exit rule	Thresholds	Test acc. (%)	Mean GFLOPs	Saved (%)	Exits 1/2/3/4 (%)
Max-prob	global	71.27 $\pm$ 0.06	0.449 $\pm$ 0.009	37.5	38 / 15 / 6 / 41
Max-prob	per-exit	71.10 $\pm$ 0.07	0.373 $\pm$ 0.034	48.0	40 / 22 / 27 / 10
Max-prob	relaxed	70.63 $\pm$ 0.26	0.357 $\pm$ 0.015	50.3	55 / 16 / 6 / 24
Entropy	global	71.23 $\pm$ 0.03	0.545 $\pm$ 0.047	24.1	24 / 10 / 4 / 62
Entropy	per-exit	71.10 $\pm$ 0.11	0.431 $\pm$ 0.056	40.1	27 / 23 / 33 / 17
Entropy	relaxed	70.38 $\pm$ 0.16	0.382 $\pm$ 0.011	46.8	50 / 15 / 6 / 28
Max-prob + TS	global	71.22 $\pm$ 0.11	0.427 $\pm$ 0.017	40.6	44 / 13 / 4 / 39
Max-prob + TS	per-exit	71.10 $\pm$ 0.05	0.345 $\pm$ 0.030	52.0	48 / 20 / 24 / 8
Max-prob + TS	relaxed	70.56 $\pm$ 0.28	0.333 $\pm$ 0.019	53.6	61 / 14 / 4 / 21
Entropy + TS	global	71.27 $\pm$ 0.06	0.461 $\pm$ 0.009	35.9	39 / 12 / 3 / 46
Entropy + TS	per-exit	71.13 $\pm$ 0.08	0.389 $\pm$ 0.039	45.9	40 / 19 / 26 / 15
Entropy + TS	relaxed	70.40 $\pm$ 0.32	0.376 $\pm$ 0.016	47.7	55 / 12 / 3 / 30

At the main operating point (no accuracy loss on validation) max-prob exiting reaches $75.32 \pm 0.12\%$ with 0.542 ± 0.037 GFLOPs, 51.2% fewer than ResNet-18; 65% of test images leave at the first exit and only 10% need the full network. On test this point is 0.8 points below the model's own final exit (and 0.7 below ResNet-18): a threshold that loses nothing on 5,000 validation images loses a little on the test set, a sign of the noise in threshold selection. Allowing one point on validation lowers the cost to 0.473 GFLOPs (57% fewer) at $74.21 \pm 0.16\%$, and fully matching the baseline's test accuracy needs 0.580 GFLOPs (48% fewer, an optimistic figure since that threshold is chosen on test). The calibrated rules (Model 3) are more conservative on clean data: max-prob with temperature scaling needs 0.613 GFLOPs and entropy with temperature scaling 0.693 GFLOPs, for slightly higher accuracy ($75.62 \pm 0.17\%$ and $75.70 \pm 0.29\%$), because scaling softens the overconfident exits so that fewer images stop early.

For EE-ViT-Tiny the picture is reversed: its exits are underconfident, temperature scaling sharpens them, and max-prob with scaling is the cheapest global rule (0.427 GFLOPs, 41% fewer than the full ViT, against 37% for raw max-prob).

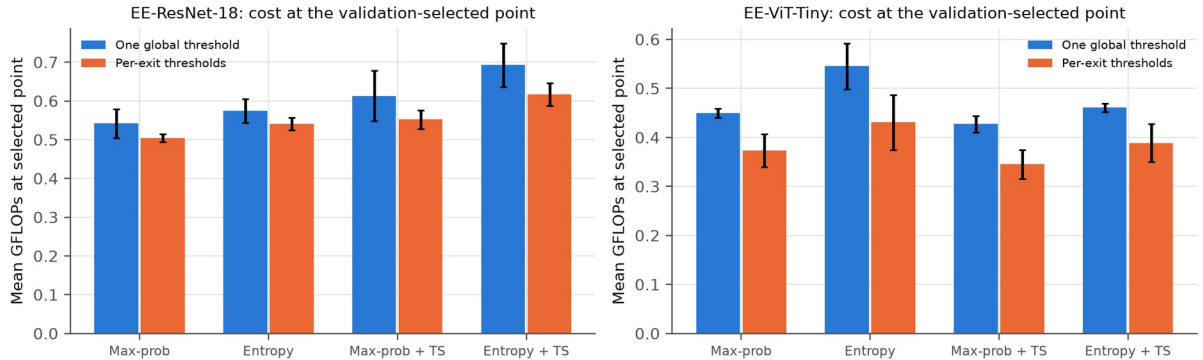


Figure 5: Mean GFLOPs at the main operating point (no validation loss) with one global threshold and with per-exit thresholds.

Per-exit thresholds lower the cost of every rule under the same validation constraint: for max-prob from 0.542 to 0.504 GFLOPs, and for entropy with temperature scaling from 0.693 to 0.616 GFLOPs; for the ViT with temperature scaling from 0.427 to 0.345 GFLOPs. A global threshold assumes that a given

confidence means the same at every exit; per-exit thresholds drop that assumption and optimise the cost directly, which temperature scaling alone does not do.

Table 9: Best test accuracy (%) reachable within a mean compute budget, as a share of ResNet-18 FLOPs (CNN) or of the full ViT (transformer).

Model	Exit rule	35% budget	50% budget	65% budget	80% budget
EE-ResNet-18	Max-prob	71.45 $\pm$ 0.29	75.62 $\pm$ 0.26	76.43 $\pm$ 0.28	76.43 $\pm$ 0.28
EE-ResNet-18	Entropy	71.18 $\pm$ 0.29	75.29 $\pm$ 0.33	76.38 $\pm$ 0.27	76.40 $\pm$ 0.28
EE-ResNet-18	Max-prob + TS	71.22 $\pm$ 0.31	75.18 $\pm$ 0.32	76.27 $\pm$ 0.24	76.34 $\pm$ 0.30
EE-ResNet-18	Entropy + TS	71.06 $\pm$ 0.37	74.25 $\pm$ 0.23	75.88 $\pm$ 0.35	76.25 $\pm$ 0.33
EE-ViT-Tiny	Max-prob	68.49 $\pm$ 0.12	70.62 $\pm$ 0.26	71.32 $\pm$ 0.08	71.33 $\pm$ 0.07
EE-ViT-Tiny	Entropy	68.37 $\pm$ 0.20	70.16 $\pm$ 0.26	71.09 $\pm$ 0.07	71.30 $\pm$ 0.08
EE-ViT-Tiny	Max-prob + TS	68.61 $\pm$ 0.06	70.85 $\pm$ 0.28	71.30 $\pm$ 0.09	71.34 $\pm$ 0.08
EE-ViT-Tiny	Entropy + TS	68.10 $\pm$ 0.18	70.16 $\pm$ 0.29	71.29 $\pm$ 0.08	71.33 $\pm$ 0.06

6.4 Ablations

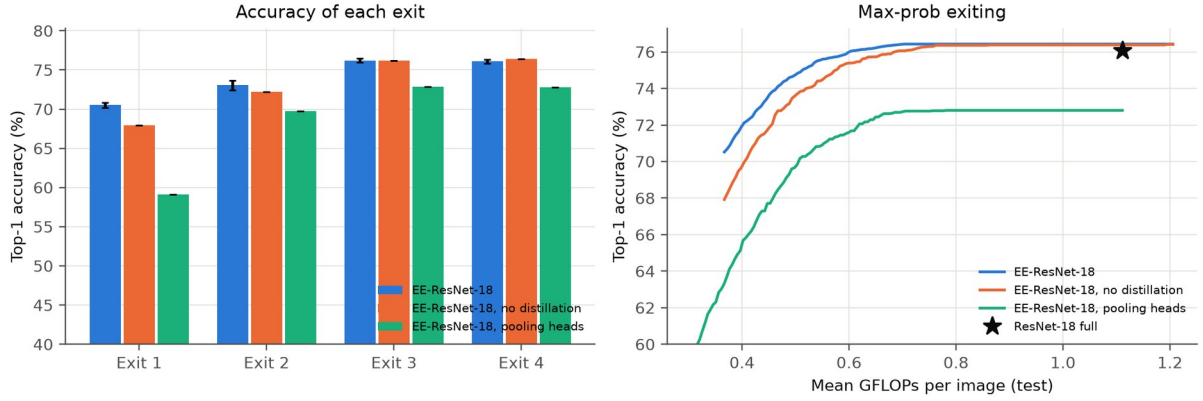


Figure 6: Effect of self-distillation and of the exit-head design: accuracy of each exit (left) and max-prob accuracy-compute curves (right).

Table 10: Ablation of training objective and head design (max-prob at the selected threshold).

Variant	Exit 1 acc.	Final acc.	Full-path GFLOPs	Max-prob acc.	Max-prob GFLOPs	Final ECE (%)
EE-ResNet-18 (Models 2, 3)	70.52 $\pm$ 0.32	76.08 $\pm$ 0.25	1.205	75.32 $\pm$ 0.12	0.542 $\pm$ 0.037	15.5
EE-ResNet-18, no distillation	67.92	76.41	1.205	75.80	0.652	13.1
EE-ResNet-18, pooling heads	59.13	72.77	1.111	72.74	0.840	17.2

Table 11: Exit placement: EE-ResNet-18 using only a subset of its trained exits (max-prob, threshold selected on validation; unused heads are not evaluated).

Exits used	Test acc. (%)	Mean GFLOPs	FLOPs saved (%)
E1+E2+E3+E4	75.32 $\pm$ 0.12	0.542 $\pm$ 0.037	51.2
E1+E2+E4	75.32 $\pm$ 0.12	0.561 $\pm$ 0.040	49.5
E1+E3+E4	75.57 $\pm$ 0.21	0.575 $\pm$ 0.017	48.3
E1+E4	75.61 $\pm$ 0.19	0.631 $\pm$ 0.023	43.2
E2+E3+E4	75.90 $\pm$ 0.10	0.734 $\pm$ 0.055	33.9
E2+E4	75.91 $\pm$ 0.11	0.768 $\pm$ 0.057	30.9
E3+E4	76.20 $\pm$ 0.30	0.856 $\pm$ 0.005	22.9
E4	76.08 $\pm$ 0.25	1.111	0.0

Self-distillation raises exit 1 from 67.92% to 70.52 $\pm$ 0.32% while the final exit changes from 76.41% to 76.08 $\pm$ 0.25%, and it lowers the max-prob cost at the main operating point from 0.652 to 0.542 GFLOPs (17% less). As Section 6.8 shows, this comes at a price under corruption. Pooling-only heads are cheaper (full path 1.111 instead of 1.205 GFLOPs) but exit 1 drops to 59.13% and even the final exit falls to 72.77%: forcing linearly separable features after the first stages hurts the shared backbone. Its main operating point needs 0.840 GFLOPs for 72.74%, so heads that can process their features are clearly worth their cost. Using all four exits is the cheapest configuration (0.542 GFLOPs). Dropping exit 1 raises the cost to 0.734 GFLOPs, while keeping only exit 1 and the final exit costs 0.631 GFLOPs: the first exit

provides most of the savings and the intermediate exits catch images that are nearly but not quite confident at exit 1.

6.5 Calibration

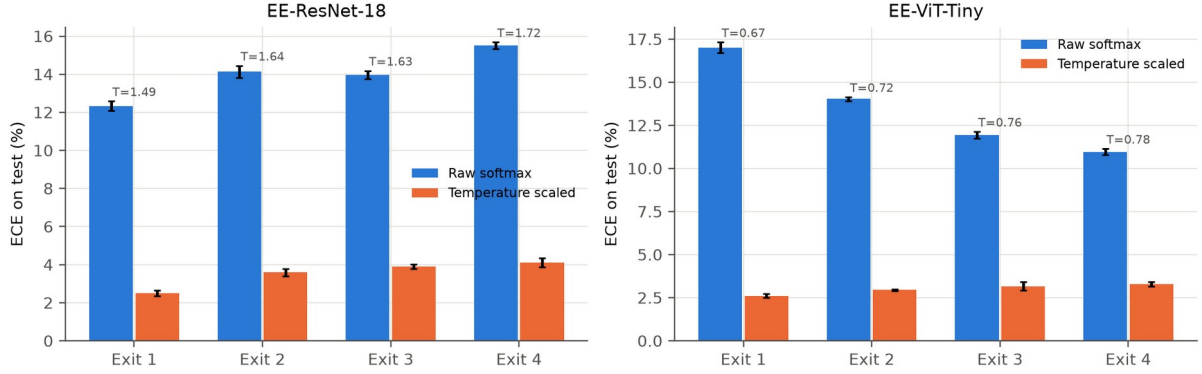


Figure 7: ECE of each exit before and after temperature scaling (mean over seeds; temperatures above the bars).

Every exit of the self-distilled CNN is overconfident: raw ECE ranges from 12.3% to 15.5%, with temperatures between 1.49 and 1.72. Training longer and distilling made even exit 1 overconfident (2.6% ECE in the 80-epoch interim model, 8.8% without distillation). Temperature scaling brings every exit to between 2.5% and 4.1%. The ViT exits are underconfident (temperatures 0.67 to 0.78), most of all the shallow ones; scaling sharpens them and reduces their ECE to at most 3.3%.

6.6 Overthinking

Table 12: Overthinking: an oracle that stops each image at its earliest correct exit.

Model	Final exit acc. (%)	Oracle acc. (%)	Oracle GFLOPs	Correct early, wrong at end (%)
EE-ResNet-18 (Models 2, 3)	76.08 $\pm$ 0.25	84.67 $\pm$ 0.31	0.549 $\pm$ 0.003	8.59 $\pm$ 0.26
EE-ResNet-18, no distillation	76.41	84.79	0.561	8.38
EE-ResNet-18, pooling heads	72.77	80.97	0.535	8.20
EE-ViT-Tiny (Model 4)	71.32 $\pm$ 0.05	76.94 $\pm$ 0.09	0.329 $\pm$ 0.000	5.62 $\pm$ 0.14

An oracle that stops each image at its earliest correct exit would reach $84.67 \pm 0.31\%$ with only 0.549 ± 0.003 GFLOPs. On $8.59 \pm 0.26\%$ of test images an early exit is correct while the final exit is wrong, so deeper processing sometimes destroys a correct answer, the overthinking effect of [4]. The gap between the oracle and every practical rule shows that deciding when to stop, not the quality of the early classifiers, is the main limitation.

6.7 Measured Latency

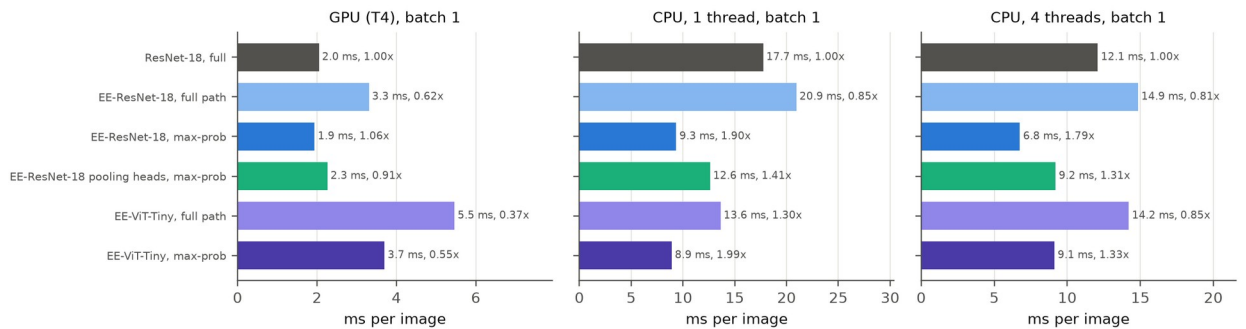


Figure 8: Measured time per image at batch 1 (seed-0 models, one machine) and speed-up over full ResNet-18 on the same device.

Table 13: Batch-1 latency per image: forced exits and adaptive inference at the selected thresholds. CPU-1 and CPU-4 use 1 and 4 threads.

Model	Device	Forced exit 1/2/3/4 (ms)	Max-prob (ms)	Entropy + TS (ms)
ResNet-18	GPU	2.05	-	-
ResNet-18	CPU-1	17.74	-	-

Model	Device	Forced exit 1/2/3/4 (ms)	Max-prob (ms)	Entropy + TS (ms)
ResNet-18	CPU-4	12.09	-	-
EE-ResNet-18	GPU	0.99 / 1.85 / 2.69 / 3.31	1.94	2.42
EE-ResNet-18	CPU-1	5.40 / 9.57 / 14.49 / 20.95	9.35	12.24
EE-ResNet-18	CPU-4	3.96 / 7.65 / 10.91 / 14.85	6.76	8.75
EE-ResNet-18, pooling heads	GPU	0.70 / 1.47 / 2.14 / 2.86	2.26	2.62
EE-ResNet-18, pooling heads	CPU-1	3.92 / 7.80 / 11.65 / 17.84	12.61	14.23
EE-ResNet-18, pooling heads	CPU-4	2.94 / 5.79 / 9.25 / 12.94	9.20	10.67
EE-ResNet-18, no distillation	GPU	0.99 / 1.90 / 2.69 / 3.42	2.13	2.52
EE-ResNet-18, no distillation	CPU-1	5.57 / 10.33 / 14.78 / 21.59	11.27	13.51
EE-ResNet-18, no distillation	CPU-4	4.40 / 7.67 / 11.51 / 15.09	8.18	9.48
EE-ViT-Tiny	GPU	1.43 / 2.80 / 4.16 / 5.46	3.70	3.91
EE-ViT-Tiny	CPU-1	3.53 / 6.91 / 10.17 / 13.62	8.92	9.00
EE-ViT-Tiny	CPU-4	3.73 / 7.08 / 10.82 / 14.21	9.12	9.69

Table 14: GPU throughput with batches of 256; exited images are removed from the batch.

Model	Full network (img/s)	Max-prob (img/s)	Entropy + TS (img/s)
ResNet-18	4735	-	-
EE-ResNet-18	4127	6493	5549
EE-ResNet-18, pooling heads	4408	5199	4778
EE-ResNet-18, no distillation	4253	6074	5499
EE-ViT-Tiny	3065	4695	4497

On one CPU thread, ResNet-18 takes 17.7 ms per image and EE-ResNet-18 with max-prob 9.3 ms, a 1.90x speed-up; with four threads it is 1.79x. On the T4 at batch 1, the baseline takes 2.05 ms and the early-exit model 1.94 ms (1.06x): small kernels and a GPU-to-CPU synchronisation after every exit dominate, so the FLOP savings barely appear. With batches of 256 and exited images removed from the batch, GPU throughput rises by a factor of 1.37. Early exits therefore pay off for CPU and edge inference, while on GPUs they need batching or kernels that avoid per-exit synchronisation. EE-ViT-Tiny shows the same split more strongly: on one CPU thread its operating point takes 8.9 ms against 13.6 ms for its full path, but on the GPU at batch 1 even the operating point takes 3.70 ms, slower than full ResNet-18, because twelve transformer blocks launch many small kernels; four CPU threads do not help its small matrix products (9.1 ms).

6.8 Robustness under Corruptions

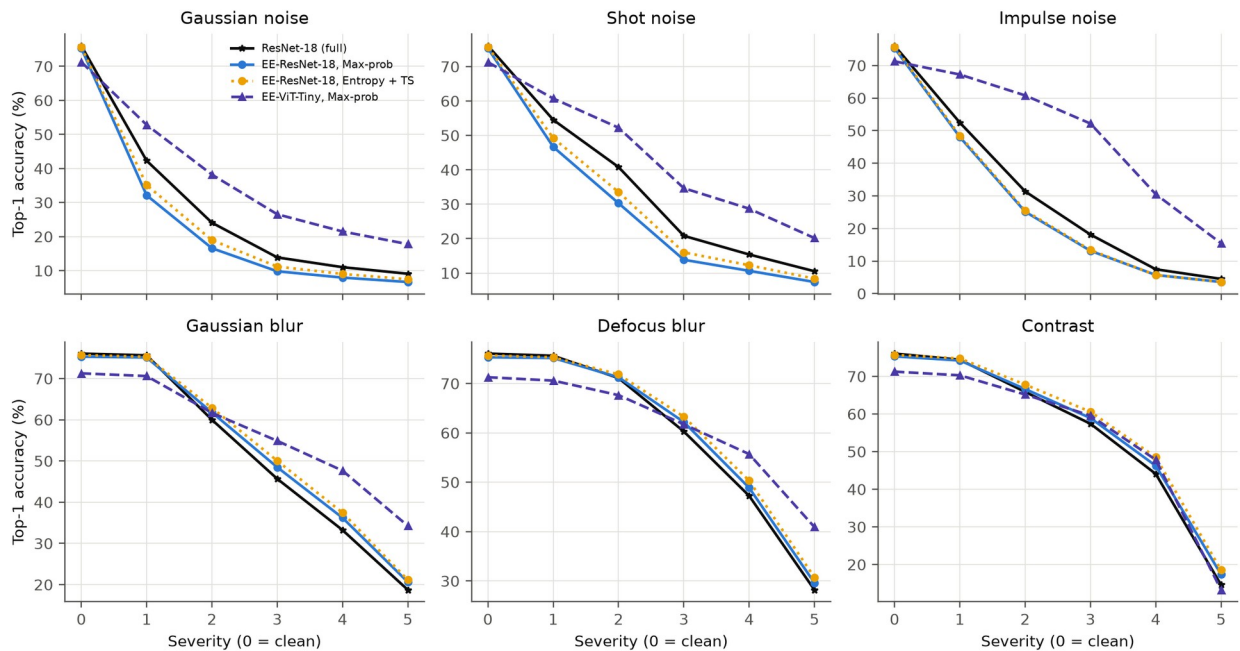


Figure 9: Accuracy against corruption severity for each corruption (mean over seeds; thresholds chosen on clean validation data).

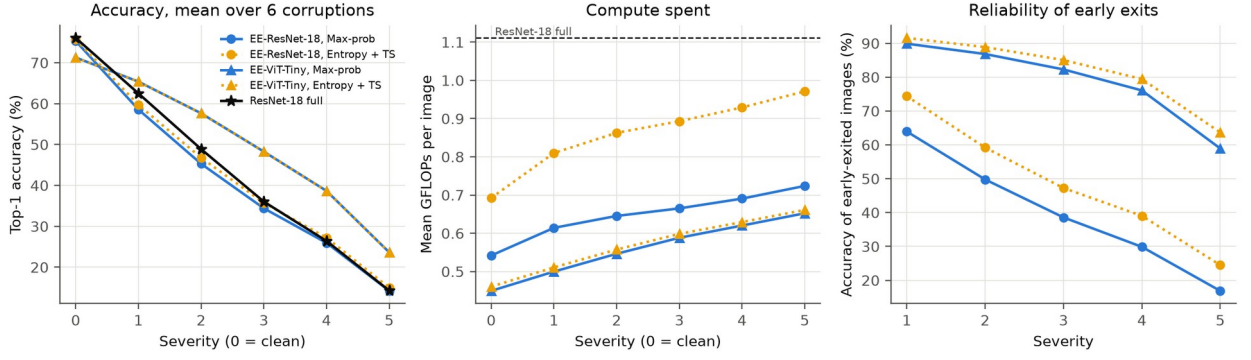


Figure 10: Mean over the six corruptions: accuracy (left), computation (middle) and accuracy of the images that exit early (right).

Table 15: Accuracy (%), mean GFLOPs and accuracy of early-exited images (%) for EE-ResNet-18 with max-prob (MP) and entropy with temperature scaling (E+TS), severities 3 and 5, mean over seeds.

Corruption	Sev.	ResNet-18 acc.	MP acc.	MP GFLOPs	MP early acc.	E+TS acc.	E+TS GFLOPs	E+TS early acc.
Gaussian noise	3	13.9	9.8	0.61	10.0	11.2	0.85	11.7
Gaussian noise	5	9.1	6.7	0.63	6.8	7.5	0.86	8.2
Shot noise	3	20.8	13.8	0.60	13.9	16.0	0.82	16.1
Shot noise	5	10.5	7.4	0.59	7.5	8.4	0.81	8.6
Impulse noise	3	18.1	13.1	0.77	15.5	13.4	1.03	20.8
Impulse noise	5	4.5	3.6	0.72	3.9	3.6	0.99	4.8
Gaussian blur	3	45.7	48.5	0.70	56.1	50.1	0.93	72.3
Gaussian blur	5	18.7	20.6	0.78	25.2	21.1	1.04	37.1
Defocus blur	3	60.3	62.2	0.63	69.3	63.3	0.84	81.8
Defocus blur	5	28.2	29.6	0.76	35.7	30.7	1.02	51.2
Contrast	3	57.4	58.9	0.67	66.7	60.6	0.90	80.8
Contrast	5	14.6	17.4	0.86	22.2	18.5	1.11	37.1

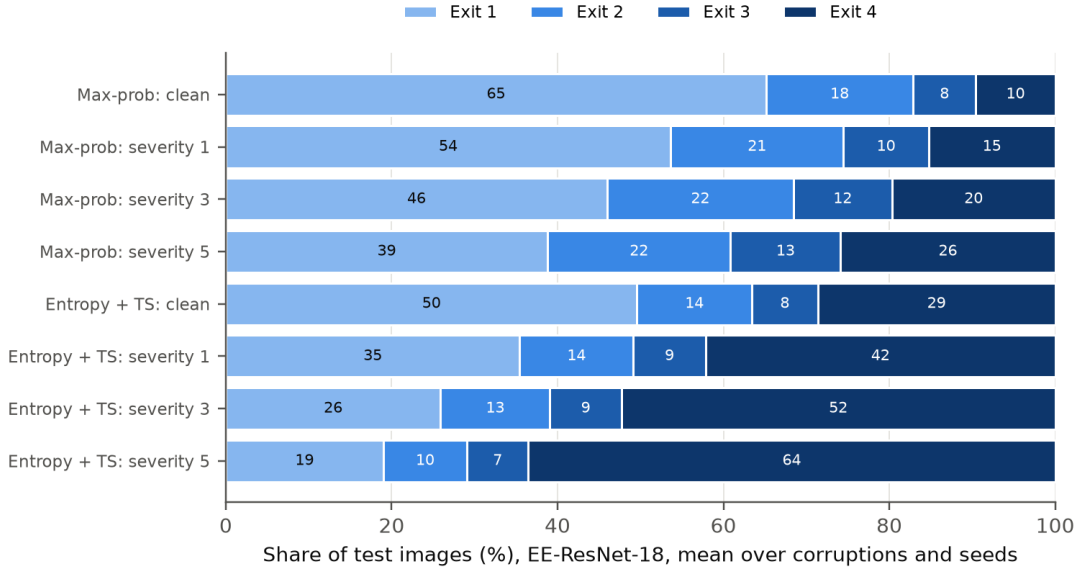


Figure 11: Where EE-ResNet-18 images exit on clean and corrupted inputs.

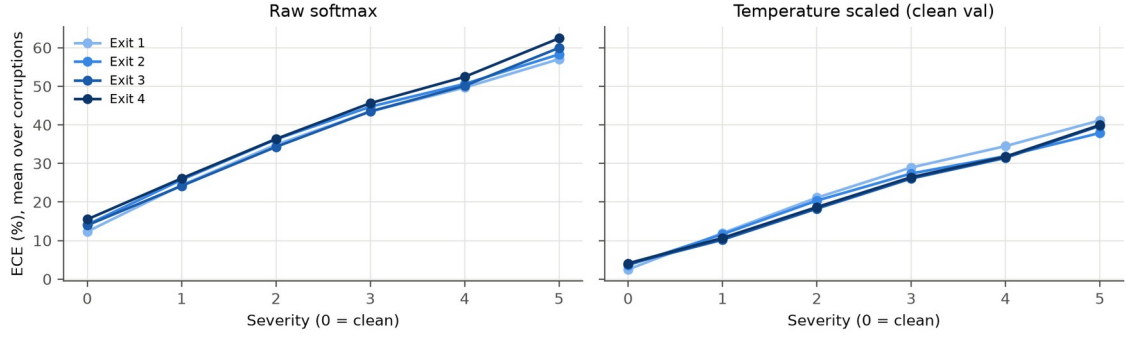


Figure 12: ECE of the EE-ResNet-18 exits under corruption, raw and with temperatures fitted on clean validation data.

With thresholds frozen from clean validation data, max-prob exiting loses accuracy relative to ResNet-18 on mildly corrupted inputs (58.6% against 62.5% at severity 1, a larger gap than on clean data) and converges to it at high severity (14.2% against 14.3%). Its computation rises only slowly, from 0.54 GFLOPs on clean data to 0.72 at severity 5, because the overconfident exits keep stopping images: 85%, 80% and 74% of images exit early at severities 1, 3 and 5, and only 17% of those at severity 5 are correct. Under Gaussian noise at severity 5, 83% of images exit early with 7% accuracy. Entropy with temperature scaling is markedly more cautious: 36% exit early at severity 5, 25% of them correctly, it spends 0.97 GFLOPs and is at least as accurate as max-prob at every severity. Self-distillation amplifies the problem: the model trained without it exits 48% of images early at severity 5 with 21% accuracy. The ViT behaves very differently. Although less accurate on clean data, it is more robust (48.3% against 36.0% for ResNet-18 at severity 3), and its underconfident exits become cautious by themselves: the share exiting early falls to 17% at severity 5, and 59% of those are correct. Temperatures fitted on clean data do not transfer: the final exit's ECE is 10.7% after scaling at severity 1 and 40.0% at severity 5 (raw: 26.1% and 62.5%).

7. Discussion

7.1 Adaptive depth (Objective 1)

The early-exit networks choose their depth per image: on clean data 65% of images stop at the first exit of EE-ResNet-18 and only 10% use the whole network. Under corruption more images continue, but far fewer than the drop in accuracy would justify for the overconfident CNN, whereas the ViT shifts most images to deeper exits.

7.2 Accuracy, computation and latency (Objective 2)

With no accuracy loss on validation, the early-exit ResNet-18 uses 51% fewer FLOPs than ResNet-18 (55% with per-exit thresholds) at $75.32 \pm 0.12\%$ against $76.07 \pm 0.31\%$. Measured latency follows FLOPs on a CPU (1.90x with one thread) but hardly on a GPU at batch 1 (1.06x), so FLOP counts alone are not a reliable proxy for speed.

7.3 Reliability under corruption (Objective 3)

Raw confidence is a poor routing signal under distribution shift: the self-distilled exits keep stopping corrupted images that they misclassify, and temperatures fitted on clean data only partly correct this. Calibrated entropy makes early decisions clearly more reliable at the cost of extra computation, removing distillation helps, and the underconfident ViT exits are the most dependable.

7.4 Relation to Prior Work

Our savings are in line with SDN [4], which reported more than 50% fewer FLOPs at equal accuracy on CIFAR-100, and our CPU speed-ups resemble the measured speed-ups of BranchyNet [1]. The missing GPU speed-up at batch 1 echoes BlockDrop's observation that sequential decisions can cost more than they save [6]. Like Meronen et al. [10] we find early exits miscalibrated; calibration improved routing for the underconfident ViT but not for the overconfident CNN on clean data, which we attribute to the threshold being shared across exits. Self-distillation, used by LGViT [9] to strengthen early exits,

improved efficiency here but made the exits overconfident under corruption, an effect that the reviewed early-exit methods do not evaluate [7].

7.5 Limitations

All experiments use CIFAR-100 at 32x32 resolution with one CNN and one transformer; larger images and other architectures may behave differently. The corruptions re-implement 6 of the 19 CIFAR-100-C types. Latency was measured on shared cloud machines (one T4 and a virtual CPU), averaged over many images but within one session. The ViT is trained from scratch and remains weaker than the CNN, the ablations use a single seed, and the 5,000-image validation set makes threshold selection somewhat noisy, as the gap between validation and test accuracy shows.

8. Conclusion and Future Work

Confidence-based early exits adapt computation to input difficulty: on CIFAR-100 an early-exit ResNet-18 keeps the accuracy of its own final exit on validation with 51-55% fewer FLOPs than ResNet-18, and self-distillation with processing heads lets most images stop at the first exit. The savings are real on CPUs but need batching to appear on GPUs. Calibration fixes the probability estimates of each exit, but routing efficiency and routing reliability are different goals: per-exit thresholds and distillation serve the first, while calibrated entropy and less overconfident exits serve the second, which matters most when the inputs drift away from the training distribution.

- Exit rules and calibration that remain reliable under distribution shift, for example corruption-aware or test-time calibration, or thresholds that adapt to the input distribution.
- Distillation objectives that strengthen early exits without making them overconfident.
- Reducing per-exit synchronisation on GPUs (batched exit decisions, fused kernels or CUDA graphs).
- Evaluation on larger datasets (Tiny-ImageNet, ImageNet subsets), the full CIFAR-100-C benchmark and pre-trained transformers, with comparisons to learned and patience-based exit policies.

References

- [1] S. Teerapittayanon, B. McDanel, and H. T. Kung, "BranchyNet: Fast inference via early exiting from deep neural networks," in Proc. 23rd Int. Conf. Pattern Recognit. (ICPR), 2016, pp. 2464-2469, doi: 10.1109/ICPR.2016.7900006.
- [2] G. Huang, D. Chen, T. Li, F. Wu, L. van der Maaten, and K. Q. Weinberger, "Multi-scale dense networks for resource efficient image classification," in Proc. Int. Conf. Learn. Represent. (ICLR), 2018, arXiv:1703.09844.
- [3] X. Wang, F. Yu, Z.-Y. Dou, T. Darrell, and J. E. Gonzalez, "SkipNet: Learning dynamic routing in convolutional networks," in Proc. Eur. Conf. Comput. Vis. (ECCV), Springer, 2018, pp. 420-436, doi: 10.1007/978-3-030-01261-8_25.
- [4] Y. Kaya, S. Hong, and T. Dumitras, "Shallow-deep networks: Understanding and mitigating network overthinking," in Proc. 36th Int. Conf. Mach. Learn. (ICML), PMLR vol. 97, 2019, pp. 3301-3310.
- [5] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in Proc. 34th Int. Conf. Mach. Learn. (ICML), PMLR vol. 70, 2017, pp. 1321-1330.
- [6] Z. Wu, T. Nagarajan, A. Kumar, S. Rennie, L. S. Davis, K. Grauman, and R. Feris, "BlockDrop: Dynamic inference paths in residual networks," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2018, pp. 8817-8826, doi: 10.1109/CVPR.2018.00919.
- [7] D. Hendrycks and T. Dietterich, "Benchmarking neural network robustness to common corruptions and perturbations," in Proc. Int. Conf. Learn. Represent. (ICLR), 2019, arXiv:1903.12261.
- [8] M. Wolczyk, B. Wojcik, K. Balazy, I. T. Podolak, J. Tabor, M. Smieja, and T. Trzcinski, "Zero time waste: Recycling predictions in early exit neural networks," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 34, 2021, pp. 2516-2528.
- [9] G. Xu, J. Hao, L. Shen, H. Hu, Y. Luo, H. Lin, and J. Shen, "LGViT: Dynamic early exiting for accelerating vision transformer," in Proc. 31st ACM Int. Conf. Multimedia (MM), 2023, pp. 9103-9114, doi: 10.1145/3581783.3611762.
- [10] L. Meronen, M. Trapp, A. Pilzer, L. Yang, and A. Solin, "Fixing overconfidence in dynamic neural networks," in Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis. (WACV), 2024, pp. 2668-2678, doi: 10.1109/WACV57701.2024.00266.
- [11] A. Krizhevsky, "Learning multiple layers of features from tiny images," Tech. Rep., Univ. of Toronto, 2009.
- [12] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2016, pp. 770-778.
- [13] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in Proc. Int. Conf. Learn. Represent. (ICLR), 2021.

- [14] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jegou, "Training data-efficient image transformers and distillation through attention," in Proc. 38th Int. Conf. Mach. Learn. (ICML), PMLR vol. 139, 2021, pp. 10347-10357.
- [15] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, "mixup: Beyond empirical risk minimization," in Proc. Int. Conf. Learn. Represent. (ICLR), 2018.
- [16] S. Yun, D. Han, S. J. Oh, S. Chun, J. Choe, and Y. Yoo, "CutMix: Regularization strategy to train strong classifiers with localizable features," in Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV), 2019, pp. 6023-6032.

Appendix A: Reproducing the Results

All code is in the repository listed on the first page. Training and evaluation run on Kaggle GPUs; analysis and report generation run on a CPU.

```

pip install -r requirements.txt
python kaggle/launch.py push final-a          # 2 x T4: one queue of runs per GPU
python kaggle/launch.py push final-b
python kaggle/launch.py push latency          # loads seed-0 checkpoints of both sessions
python kaggle/launch.py fetch final-a && python kaggle/launch.py fetch final-b
python kaggle/launch.py fetch latency
python scripts/analyze_final.py               # results/final/
python report/build_report.py final           # report/Final_Report.docx and .pdf

```

Table 16: Repository layout.

Path	Contents
src/data.py, src/corruptions.py	CIFAR-100 split, GPU augmentation, RandAugment, six corruptions
src/models/	MultiExitNet, ResNet-18 variants, ViT-Tiny
src/train.py	multi-exit training with self-distillation
src/exits.py	exit scores, temperature scaling, ECE, routing, threshold selection and tuning
src/evaluate.py, src/latency.py	logits on clean and corrupted data, latency benchmark
kaggle/	Kaggle job template and launcher
scripts/analyze_final.py	final analysis and figures
report/	report generators and outputs